BRI-410: A Chess-Inspired Tweakable Block Cipher with a 410-bit Key, Designed by Measurement

Rinald Cobaj, Independent researcher, vlorarino@gmail.com

Draft, 2026-10-22. Study cipher: it has not yet been analysed by independent cryptanalysts and must not be used to protect real data.

Abstract

We present BRI-410, a tweakable block cipher with a 512-bit block, a 410-bit key, a 128-bit tweak and 16 rounds, together with BRI-410 AE, an authenticated-encryption mode based on ΘCB3 with a 416-bit tag. The design started from an analogy with chess: the rules of the game are public while every game played is unique. The game-tree complexity of chess, about 10^123 ≈ 2^410, fixes the key length. The 8×8 board fixes the state, and the knight's move gives a key- and tweak-dependent cell permutation. The idea that *every game is played only once* led to deriving all round keys and the permutation from the key **and** a per-block tweak. The cipher is a key-alternating SPN with an AES-like 8-bit S-box (differential uniformity 4, nonlinearity 112, degree 7) and an 8×8 Cauchy MDS matrix over GF(2^8).

The design was driven by measurements. An earlier version with 4-bit cells (BRI-410/4) had a 4-round integral distinguisher with 2^12 chosen plaintexts and a practical 5-round key-recovery attack. It was replaced. For the final version we prove with a CP-SAT model that, for every choice of the key-dependent permutations, any 4-round differential or linear trail has at least 81 active S-boxes (probability ≤ 2^-486) and any 6-round trail at least 90 (≤ 2^-540). The best attacks we found reach 4 of 16 rounds in practice, with an estimated 5-round attack at about 2^72. We state explicit security claims, list open problems and propose reduced-round challenges.

1. Introduction and motivation

The rules of chess are known to everyone, yet a serious game almost never repeats. Shannon estimated the number of games at about 10^120 [Sha50]. Allis estimated the game-tree complexity at about 10^123 [All94]. BRI-410 turns this into a design principle:

Chess	BRI-410
public rules	public algorithm (Kerckhoffs' principle)
a game among ≈ 10^123	a key among 2^410 ≈ 10^123.4
the 8×8 board	the 512-bit state: 64 cells of 8 bits
the knight's move	the cell permutation π
every game is played only once	every block uses its own round keys and permutations (tweak)

The project followed a fixed rule: no design choice is accepted without a measurement. Every weakness found during the design is reported in Section 5, together with the change it caused.

2. Specification of BRI-410/8

2.1 Notation and parameters

Parameter	Value
Block	512 bits = 64 bytes. Byte i is cell (r, k) = (⌊i/8⌋, i mod 8) of an 8×8 board
Key	410 bits, stored as 52 bytes little-endian; the 6 top bits must be 0
Tweak	128 bits (16 bytes)
Rounds	R = 16

Arithmetic on cells is in GF(2^8) with the polynomial x^8 + x^4 + x^3 + x + 1.

2.2 Round function

```
X ← X ⊕ RK_0
for i = 1 .. 16:   X ← θ(π_i(S(X))) ⊕ RK_i
```

All rounds are identical, and the last round also applies θ. Decryption applies `⊕RK_i, θ^-1, π_i^-1, S^-1` for i = 16..1, then `⊕RK_0`.

S (S-box). `S(x) = A·x^-1 ⊕ c` (with `0^-1 = 0`), where `A` is an invertible 8×8 binary matrix (row i is a byte; bit i of `A·y` is the parity of `A[i] AND y`) and `c` is a byte. `A ‖ c` are the 9 bytes of `SHAKE-256("BRI-410 S-box8 v1" ‖ n)`, for the first 4-byte big-endian counter `n` that gives an invertible `A` and no points with `S(x) = x` or `S(x) = x ⊕ FF`. This gives `n = 9`. As an affine transform of the inverse, S has differential uniformity 4, linearity 32 (nonlinearity 112) and algebraic degree 7 in every component.

π (the knight's move). Cell (r, k) makes k knight jumps (dr, dk) on a toroidal board, then the board shifts β columns: `(r, k) → (r + k·dr, k + k·dk + β) mod 8`, with (dr, dk) ∈ {(1,2), (1,−2), (−1,2), (−1,−2)} and β ∈ {0..7}. All 32 permutations map the 8 cells of every row to 8 different rows ("diffusion-optimal"). A row shift α was removed during the design because it commutes with the whole unkeyed round, so it would add no structure (Section 5.2).

θ (MDS). Each row of 8 bytes is multiplied by `M[i][j] = 1 / (x_i ⊕ y_j)`, `x = 0..7`, where `y` are the first 8 distinct bytes not in {0..7} of `SHAKE-256("BRI-410 MDS8 v1")`, namely `y = [145, 234, 150, 206, 98, 226, 235, 94]`. M is MDS (verified: none of its 12 869 square submatrices is singular), so its branch number is 9. It has no column symmetry and is not involutory.

2.3 Key and tweak schedule ("the game")

The schedule state has 80 bytes (10 rows of 8). It is loaded with the key in bits 0..409 and the 230 low bits of `SHAKE-256("BRI-410/8 mbushja")` (29 bytes, little-endian) in bits 410..639. A step `F_i` is:

```
S on all 80 bytes → θ on the 10 rows → byte j moves to position 9·j mod 80 → byte 79 ⊕= i
```

Tweak injection: `F_0xF0 … F_0xF3` (key only), then bytes 0..15 ⊕= tweak, then `F_0xF4 … F_0xF7`. Then `RK_0` = bytes 0..63. For i = 1..16: apply `F_i`, `RK_i` = bytes 0..63, and the permutation of round i is taken from byte 64 = b: direction = list[b & 3], β = (b >> 2) & 7.

The first four steps depend only on the key and can be computed once per key.

3. BRI-410 AE

BRI-410 AE is the ΘCB3 construction of Krovetz and Rogaway [KR11] with BRI-410/8 as the tweakable block cipher E. Every call uses the tweak `N ‖ ((index << 4) | type)` (12-byte nonce, then 4 bytes little-endian, index < 2^28). The types are: 0 message block, 1 mask of the final partial block, 2 tag without partial block, 3 tag with partial block, 4 associated-data block, 5 final partial associated-data block. Associated data uses `N = 0^12`. With `M = M_1 … M_m M_*` and `A = A_1 … A_a A_*` (|M_*|, |A_*| < 64 bytes) and `pad(x) = x ‖ 80 ‖ 00…`:

```
C_i  = E^(N,i,0)(M_i)                                          i = 1..m
if M_* ≠ ε:  C_* = M_* ⊕ E^(N,m,1)(0^512)[first |M_*| bytes];  F = E^(N,m,3)(⊕M_i ⊕ pad(M_*))
else:        F = E^(N,m,2)(⊕M_i)
Auth = ⊕ E^(0,i,4)(A_i)  [⊕ E^(0,a,5)(pad(A_*))]
Tag  = (F ⊕ Auth)[first 52 bytes]              − 416 bits
Packet = N ‖ C ‖ Tag
```

Decryption returns nothing unless the tag verifies (constant-time comparison).

4. Security claims

Claim 1 (tweakable block cipher). BRI-410/8 with 16 rounds is a secure tweakable block cipher in the single-key setting with adaptively chosen tweaks, including repeated tweaks. There is no key-recovery attack and no distinguishing attack from an ideal tweakable permutation with time complexity below 2^410 and data complexity below 2^256 queries per key.

Claim 2 (authenticated encryption). If Claim 1 holds, BRI-410 AE provides confidentiality and integrity against nonce-respecting adversaries, as in the ΘCB3 security theorem [KR11]. Each forgery attempt succeeds with probability about 2^-410 at most: the tag has 416 bits, but security is bounded by the key. Limits: at most 2^28 − 1 blocks per message. With random nonces, at most 2^32 messages per key (nonce-collision probability ≈ 2^-33).

Not claimed. Security in the related-key setting. Security of BRI-410/8 used inside a hash function (known- or chosen-key setting). Resistance of AE to nonce misuse: a repeated nonce reveals equal blocks and may allow forgeries. Side-channel resistance of the reference implementation, which is not constant-time.

5. Design rationale and analysis

All numbers can be reproduced with the scripts of the accompanying repository.

5.1 Components

Component	Measurement
S-box	DU 4, nonlinearity 112, degree 7, no fixed points (10 random 8-bit permutations: DU 10–16, NL 90–96)
θ	MDS (0/12 869 singular minors), 1 column symmetry (the identity)
π	32 distinct permutations, all diffusion-optimal
Full diffusion	every output byte depends on every input byte after 2 rounds; plaintext avalanche 50.2% after 2 rounds

5.2 History: what was measured and changed

1. **The row shift α was removed.** In the first design the knight's move also had a row shift α (3 bits). We measured that α commutes with the unkeyed round, so it can be moved to the end of the cipher. This gave 1 792 pairs of "game" variants that were equivalent up to α. After setting α = 0 there were none.
2. **A game per key was not enough. A game per block was needed.** With the permutation derived from the key only, an integral attack needs thousands of texts under the *same* permutation. Making every block use its own tweak-derived round keys and permutations removed the integral distinguishers completely. Measured on BRI-410/4, the 5-round key recovery succeeded 0/3 times (3/3 before, and 3/3 when the tweak is repeated). On BRI-410/8, the 3-round integral distinguisher also disappears with a unique tweak per block. The game itself hides only 1 bit per round at the row level: the destination row depends only on dr. Its main value is that the tweak makes every block a different permutation.
3. **4-bit cells were replaced by 8-bit cells.** BRI-410/4 (4-bit cells, 256-bit block, 32 rounds) had an integral distinguisher over 4 rounds with only 2^12 chosen plaintexts (3 cells aligned by π_1), one round more than AES with comparable data. We executed a 5-round key recovery in under one second. The likely cause is the low degree (3) of 4-bit S-boxes. With 8-bit cells the integral distinguisher stops at 3 rounds: over 4 rounds, 0–1 of 64 bytes are balanced even with 2^16 aligned texts, as for a random function.

5.3 Differential and linear cryptanalysis (CP-SAT bounds)

Word-level model: S preserves activity. θ satisfies "active inputs + active outputs of a row ∈ {0} ∪ [9,16]". For θ only the destination row given by π matters, and **the adversary may choose dr in every round**, so the bounds hold for every key and tweak. The same bounds hold for linear trails, because the transpose of an MDS matrix is MDS.

Rounds	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**16**
min. active S-boxes	1	9	17	**81**	82	90	98	162	324
proof	CP-SAT	CP-SAT	CP-SAT	CP-SAT	CP-SAT	4+2	4+3	4+4	4×4
$\log_2$ max trail probability	−6	−54	−102	**−486**	−492	**−540**	−588	−972	−1944

For 6–8 rounds, CP-SAT also found trails with exactly 90, 98 and 162 active S-boxes, so these values are tight.

Tweak schedule (related-tweak). A difference injected through the tweak must activate at least 1, 9, 17, **81**, 107, 114 S-boxes in 1..6 schedule steps (all proven by CP-SAT). With the 4 steps used, predicting the difference of RK_0 has probability ≤ 2^-486.

5.4 Experiments

Experiment	**Result**
Integral, fixed tweak, 1 byte (2^8 texts)	balanced after 2 and 3 rounds (10/10), random after 4 rounds
Integral, fixed tweak, 2 aligned bytes (2^16)	random after 4 rounds
4-round key recovery (fixed tweak)	5/5, RK_4 (512 bits) recovered with 512–768 texts
5-round key recovery (estimate)	≈ 2^72 (guess 8 + 1 bytes per target byte)
Related-tweak integral (1 byte / 2 bytes of tweak, 2^8 / 2^16 tweaks)	shortcuts only with ≤ 1 mixing step and ≤ 2 rounds. 4 steps and 16 rounds: random
Tweak avalanche (1 bit)	RK_0 changes 49.9% after 4 steps; ciphertext 49.8%
AE	all 1 312 single-bit modifications of a packet rejected; 0/2 000 random forgeries accepted

5.5 Rounds

Our rule: $R \geq 2 \times \max(R_1, R_2, R_3)$, where R_1 = rounds to full diffusion (2), R_2 = rounds until the best trail is below 2^-512 (6, proven), and R_3 = rounds attacked (4 executed, 5 estimated). This gives $R \geq 12$. We use 16 rounds.

6. Performance

Pure Python reference, single core: about 150 KB/s for long messages with BRI-410 AE. AES-256 in pure Python (our comparison code) runs at about 356 KB/s. Because the round keys are recomputed for every block (Section 2.3), BRI-410 is slower than a design that injects the tweak more lightly (e.g. TWEAKEY [JNP14]). A constant-time C implementation is an open task.

7. Open problems and challenges

Open problems.

1. Integral attacks with the division property [Tod15], for 4 rounds with ≥ 2^24 texts and beyond.
2. Clustering of differential trails and linear hulls. Impossible-differential, boomerang and algebraic attacks.

3. Related-tweak attacks that combine a tweak difference with a plaintext difference, including pairs whose permutations differ.
4. A lighter tweak schedule (TWEAKEY-style) with the same security.
5. A constant-time bitsliced implementation in C and its performance.

Challenges (the author will acknowledge every result):

Target	Fixed tweak	Chosen tweaks
distinguisher	5, 6, 7, 8 rounds	5, 6, 7, 8 rounds
round-key recovery	6, 7, 8 rounds	6, 7, 8 rounds
master-key recovery	any number of rounds	any number of rounds

8. Reference implementation and test vectors

`reference/bri410.py` is a single file that uses only the Python standard library. It contains the block cipher and BRI-410 AE. `reference/test_vectors.json` has 12 block-cipher vectors (including RK_0 and RK_16) and 6 AE vectors. `reference/verify.py` checks them. The vectors were produced by an independent implementation of the same specification, and both implementations agree. The code, the test vectors and every analysis script are available at https://github.com/rinaldcobaj-cell/BRI-410.

Intellectual property

BRI-410 is not patented, and the author does not intend to apply for any patent on it. The specification, the reference implementation, the test vectors and the analysis scripts are dedicated to the public domain under CC0 1.0 Universal. Anyone may use, analyse, implement and modify them freely.

Acknowledgements

The design ideas and decisions are the author's: the chess analogy and the 410-bit key, the per-block "game played only once", the choice of 8-bit cells and 16 rounds, and the 416-bit tag. The author used an AI model (Claude, by Anthropic) as an assistant to write the implementation and to run the experiments and the CP-SAT computations. Every result in this paper can be reproduced with the published code, and the author is responsible for its content.

References

- [All94] L. V. Allis. *Searching for Solutions in Games and Artificial Intelligence.* PhD thesis, University of Limburg, Maastricht, 1994.
- [BS91] E. Biham, A. Shamir. Differential cryptanalysis of DES-like cryptosystems. *Journal of Cryptology*, 1991.
- [DKR97] J. Daemen, L. Knudsen, V. Rijmen. The block cipher Square. *FSE 1997*.
- [DR02] J. Daemen, V. Rijmen. *The Design of Rijndael.* Springer, 2002.
- [JNP14] J. Jean, I. Nikolić, T. Peyrin. Tweaks and keys for block ciphers: the TWEAKEY framework. *ASIACRYPT 2014*.
- [KR11] T. Krovetz, P. Rogaway. The software performance of authenticated-encryption modes. *FSE 2011*.
- [LRW02] M. Liskov, R. Rivest, D. Wagner. Tweakable block ciphers. *CRYPTO 2002*.
- [Mat93] M. Matsui. Linear cryptanalysis method for DES cipher. *EUROCRYPT 1993*.
- [MWGP11] N. Mouha, Q. Wang, D. Gu, B. Preneel. Differential and linear cryptanalysis using mixed-integer linear programming. *Inscrypt 2011*.
- [Sha50] C. E. Shannon. Programming a computer for playing chess. *Philosophical Magazine*, 1950.
- [Tod15] Y. Todo. Structural evaluation by generalized integral property. *EUROCRYPT 2015*.